

It is hard enough developing embedded projects and you do not want to be struggling with OS-related problems as well. The OS must be a trusted component that the developer can rely on, supported by in-depth training and good, responsive support

also matters a lot if devices are peripheral-rich and you have future plans of upgrading the hardware," says Jukaku. For power management, the OS should be able to provide power-saving features, like suspend/wake-on-interrupts. He adds, "Some other factors include availability of software protocols and development libraries, which could be specific to the application area. The level of vendor/community support for the OS also needs to be high."

While designing an embedded system, parameters such as computing power, memory, electrical power, real-time behaviour, regulatory guidelines, connectivity, safety, security and manageability should also be considered while selecting an OS. Citing an example, Kumar says, "If you are designing a life-critical device, such as a pacemaker, then the OS would need to be deterministic, small and extremely power efficient." He adds, "If you are designing an avionics system, it would still need to be power efficient but not as much as the pacemaker. For sub-systems, meeting safety requirements outlined by the regulatory authority is one of the most important requirements and a certified/certifiable OS would be more appropriate."

For developers, real-time operating system (RTOS) selection has traditionally been a matter of preference and convenience, as they tend to look

Important parameters to consider before finalising an OS

Responsiveness. The OS-scheduling algorithm, interrupt latency and context switch times will significantly define the responsiveness and determinism of the system. The most important consideration is the kind of response desired (Is a hard real-time response required?). This means that there are precisely-defined deadlines that, if not met, result in system failure. Alternatively, would a non-deterministic, soft real-time response be appropriate? In this case, there are no guarantees to when each task will be completed.

Available system resources. Micro-kernels are designed to work using minimum system resources and provide limited but essential task-scheduling functionality. Micro-kernels generally deliver a hard real-time response, and are used extensively with embedded microprocessors with limited random access memory (RAM)/read only memory (ROM) capacity, but can also be appropriate for larger embedded processor systems.

Alternatively, a full-featured OS, like Linux or WinCE, could be used. These provide a feature-rich OS environment, normally supplied with drivers, graphical user interfaces (GUIs) and middle-ware components. Full-featured OSs are generally less responsive, require more memory and more processing power than micro-kernels, and are mainly used on powerful embedded processors where system resources are plentiful.

Middle-ware and drivers. The OS supplier should be able to provide the middle-ware and drivers you require, integrated with the OS and your target hardware. The vendor should also be in a position to provide support and ultimately take responsibility for the complete package.

Open source or professionally-licensed. There are many widely-used, free open source OSs available, distributed under general-public licence (GPL) or modified-GPL licences. However, these licences may contain copy-left restrictions and offer little protection. Professionally-licensed products remove the copy-left restrictions, offer full intellectual property (IP) infringement indemnification and warranties. In addition, you have a single company providing support and taking responsibility for the quality of the product.

Cost. A proprietary OS results in increased cost. For instance, Windows-enabled phones require licence pay-outs to Microsoft, thereby pushing up the cost of the product. Open source is free. Many companies have customised open source to a high degree and charge premium for further customisation.

Functionality levels. In terms of functions, permitting back and forth data transfer rather than only one-way would be beneficial for your embedded design. Your design is further benefited if the OS provides dynamics during run-time, supports functions to handle interrupt requests, handles multiple tasks, supports multiple platforms/architecture and is highly stable and secure. If not, it significantly increases the development time and resources required.

Flexibility. A proprietary OS licence will not permit actual alteration of the OS itself, whereas open source OSs are open to complete alteration as per the need of the hour, as many times as required. However, there may be conditions to this, such as open sourcing your changes, depending on the licence.

Quality. What emphasis does the OS supplier place on quality within his or her organisation? Quality is more than just a coding standard. Are correct procedures in place to guarantee the quality of future products and support? Well-managed companies that take quality seriously tend to be ISO 9001 certified.

Safety certification. Pre-certified and certifiable OSs are available for applications that require certification to international design standards, such as DO-178C and IEC 61508. These OSs provide key safety features, and the design evidence required by certification bodies to confirm that the process used to develop the OS meets the relevant design standard.

—Contributions from Andrew Longhurst, engineering and business development manager, Wittenstein High Integrity Systems, and Neeraj Saraf, CEO, Seal Technologies

at compatibility with their choice of compilers, debuggers and other development tools, informs Prasad Suri, AVP-sales, product engineering services, ValueLabs. He says, "Many use integrated development environments (IDEs) that enable them to develop a wider range of RTOSs."

Another critical factor for the success of a project is the selection

of an OS that ensures right time to market for the application. Suri adds, "RTOSes that offers simple system services, intuitive naming conventions, documentation, good support and availability of full source code should be preferred as these characteristics enable developers to become productive in a short period and complete projects on schedule."